

Relatório de Mapeamento, Coleta e Homologação (Fases 1 a 3)

Este relatório consolida todas as etapas técnicas executadas no projeto de migração e reestruturação do NextERP (GCP → Hostinger VPS), traduzidas para uma linguagem executiva e acessível a gestores. Ele serve como registro histórico de engenharia reversa, análise de incidentes de homologação e guia de melhoria contínua.

Sumário

1. Introdução e Objetivo do Projeto
 2. Fase 1: Investigação e Mapeamento na GCP
 - o 2.1 Restabelecimento do Acesso SSH Seguro
 - o 2.2 Descoberta da Arquitetura do Docker Compose
 - o 2.3 Análise do Servidor Web Caddy
 - o 2.4 Descoberta dos Fluxos do Extrator e Backups
 3. Fase 2: Geração de Backups e Coleta Segura
 - o 3.1 Estratégia de Dumps e Compressão Nativos
 - o 3.2 Padronização Semântica de Arquivos
 - o 3.3 Transferência de Baixo Impacto
 4. Fase 3: Homologação no Laboratório Local (openSUSE)
 - o 4.1 Ocorrência 01: Conflito de Portas de Rede (Nginx Proxy Manager)
 - o 4.2 Ocorrência 02: Ambiente Virtual Python Corrompido (PEP 668)
 - o 4.3 Ocorrência 03: Erro de Acesso Negado (Sincronização de Senhas MariaDB)
 - o 4.4 Execução Bem-Sucedida dos Extratores
 5. O Porquê desta Abordagem (Efetividade do Laboratório)
 6. Cronograma das Etapas Restantes (Hostinger VPS)
 7. Plano de Melhoria Contínua para o Ambiente
-

1. Introdução e Objetivo do Projeto

O **NextERP** é o sistema central de gestão de estoque e operações da CDC, rodando sobre a tecnologia containerizada Frappe/ERPNext v15.

O objetivo principal deste projeto é transferir o sistema do provedor antigo (**Google Cloud / GCP**) para um novo servidor dedicado na **Hostinger**. Essa mudança visa **reduzir os custos mensais de infraestrutura**, aumentar o desempenho do sistema e garantir o controle total sobre as informações da empresa.

Como o ambiente antigo foi construído sem documentação técnica pela equipe anterior, as atividades até o momento focaram em realizar uma “perícia” no servidor antigo, extrair cópias de segurança (backups) e testar o funcionamento completo em um

laboratório de simulação local (computador de desenvolvimento) antes de publicar no servidor definitivo.

2. Fase 1: Investigação e Mapeamento na GCP

2.1 Restabelecimento do Acesso SSH Seguro

O acesso inicial ao servidor antigo no GCP estava bloqueado devido a regras de chaves do Google. * **O que foi feito:** Acessamos o painel do Google Cloud e cadastramos uma chave de acesso privada e segura (id_ed25519). * **Desligamento Seguro:** Para aplicar as alterações de segurança sem corromper o banco de dados, realizamos um desligamento controlado da máquina (*Graceful Shutdown*). Na reinicialização, o GCP atribuiu o novo endereço de IP **136.113.22.112**, e o acesso seguro foi reestabelecido com sucesso.

2.2 Descoberta da Arquitetura do Docker Compose

Ao investigar o servidor antigo, localizamos os arquivos de configuração do sistema (docker-compose.yml): * **Serviços Mapeados:** Identificamos que o sistema roda dividido em contêineres isolados (Banco de dados MariaDB 10.6, Motores de fila Redis, Servidor Web Nginx e Agendadores de tarefas). * **Vulnerabilidade Encontrada:** A senha de administrador do banco de dados estava configurada como "admin" (uma senha padrão frágil que será alterada na migração). * **Isolamento:** O banco de dados e os motores de processamento estão protegidos em uma rede interna, sem exposição direta para a internet pública.

2.3 Análise do Servidor Web Caddy

Inspecionamos a porta de saída do servidor e identificamos a presença do proxy **Caddy**. * **Domínio Oficial:** Confirmamos a regra ativa apontando para o endereço **estoque.cdc.org.br**. * **Funcionamento:** O Caddy recebe os acessos dos usuários na internet e os redireciona com segurança para a aplicação ERPNext na porta interna 8080.

2.4 Descoberta dos Fluxos do Extrator e Backups

Descobrimos duas automações essenciais rodando em segundo plano no servidor antigo: * **Backup Offsite (bkp.py):** Roda duas vezes ao dia. Ele extrai uma cópia do banco de dados e envia para uma conta do Google Drive via autenticação OAuth. * **Integrador de Dados (run_job.sh):** Um pipeline composto por 5 scripts em Python que roda de hora em hora. Ele se conecta via internet segura (HTTPS porta 443) com o sistema parceiro **ONGSYS** (www.ongsys.com.br) para baixar e atualizar dados de produtos e requisições de estoque na CDC. * **Confirmação Técnica:** Verificamos que o sistema não depende de drivers de banco de dados antigos (ODBC), simplificando a instalação na Hostinger.

3. Fase 2: Geração de Backups e Coleta Segura

3.1 Estratégia de Dumps e Compressão Nativos

Utilizando os comandos oficiais do ERPNext, geramos uma cópia de segurança atômica e consistente de produção:

```
sudo docker exec -it frappe_docker-backend-1 bench --site frontend
    backup --with-files
```

Isso gerou o banco de dados SQL (10.9 MB), os arquivos públicos anexados por usuários (530 KB), os arquivos privados (40 KB) e as chaves de criptografia do sistema (site_config.json).

3.2 Padronização Semântica de Arquivos

Para evitar confusões com os nomes numéricos complexos gerados pelo sistema, renomeamos os arquivos localmente para nomes claros e autoexplicativos: * gcp-prod-database.sql.gz (Banco de Dados de Produção) * gcp-prod-public-files.tar (Arquivos Públicos) * gcp-prod-private-files.tar (Arquivos Privados) * gcp-prod-site-config.json (Configurações de Chaves)

3.3 Transferência de Baixo Impacto

Os arquivos de backup e as pastas de scripts foram copiados para uma área temporária no servidor GCP, tiveram suas permissões ajustadas para o usuário dxcdc e foram baixados com segurança via scp diretamente para o computador local.

4. Fase 3: Homologação no Laboratório Local (openSUSE)

Para garantir que nada falhasse na migração real, montamos uma réplica idêntica do ERPNext no computador local de desenvolvimento. Durante essa simulação, identificamos e corrigimos três problemas críticos que teriam causado a queda do sistema se tivéssemos ido direto para a Hostinger:

4.1 Ocorrência 01: Conflito de Portas de Rede (Nginx Proxy Manager)

- **Sintoma (O que aconteceu):** O sistema local não conseguia abrir a tela inicial porque a porta de rede 8080 já estava sendo usada por outro aplicativo do computador.
- **Causa:** Conflito de endereçamento local.
- **Solução Aplicada:** Redirecionamos a porta do ERPNext de testes para **8085** no arquivo docker-compose.yml. O sistema abriu perfeitamente em `http://localhost:8085`.

4.2 Ocorrência 02: Ambiente Virtual Python Corrompido (PEP 668)

- **Sintoma (O que aconteceu):** Os scripts do integrador falharam ao tentar instalar os pacotes necessários no computador local.
- **Causa:** A pasta de dependências (venv) foi copiada diretamente do servidor GCP (Debian) e continha atalhos internos que não funcionavam no sistema local (openSUSE).
- **Solução Aplicada:** Exluímos a pasta de dependências antiga e permitimos que o script recriasse um ambiente virtual limpo e compatível com o sistema local.

4.3 Ocorrência 03: Erro de Acesso Negado (Sincronização de Senhas MariaDB)

- **Sintoma (O que aconteceu):** A API do ERPNext retornou erro de conexão de banco de dados (Access denied for user).
- **Causa:** O banco de dados de teste criou uma senha temporária ao iniciar. Ao injetarmos o arquivo de configurações de produção (site_config.json), a senha do arquivo não batia com a senha cadastrada no banco local.
- **Solução Aplicada:** Acessamos o banco de dados e redefinimos a senha interna para coincidir com a chave oficial. A conexão foi reestabelecida imediatamente.

4.4 Execução Bem-Sucedida dos Extratores

Após as correções, executamos o pipeline do integrador de dados. Todos os 5 scripts rodaram sequencialmente com **sucesso absoluto (Código de Saída 0)**, comprovando que o banco de dados restaurado está 100% saudável.



Figura 1: Infográfico Passo a Passo do Fluxo de Homologação Executado no Laboratório Local.

5. O Porquê desta Abordagem (Efetividade do Laboratório)

Realizar esse teste completo em um laboratório isolado garantiu três grandes benefícios de governança para a CDC:

1. **Garantia de Restauração (Restore):** Provamos na prática que os arquivos de backup gerados no GCP não estão corrompidos e contêm todos os dados reais.
2. **Mitigação de Downtime (Sem Paralisação):** Todos os erros de compatibilidade, portas ocupadas e senhas desalinhadas foram resolvidos no ambiente de teste, sem afetar os funcionários da CDC que continuavam usando o sistema no GCP.
3. **Segurança de Processos:** A receita do Docker Compose e as chaves de acesso foram validadas antes de serem implantadas no servidor final da Hostinger.



Figura 2: Valoração dos Ativos de Dados e Mitigação de Riscos Operacionais da CDC.

6. Cronograma das Etapas Restantes (Hostinger VPS)

A transição final para o novo servidor da Hostinger será dividida em 8 passos planejados para garantir uma janela de manutenção segura e sem perda de informações:

Etapa	Ação Técnica	Tempo Est.	Impacto na Operação
Passo 1	Acessar e configurar o Docker/Docker Compose na Hostinger VPS.	30 min	Nenhum (sistema ativo no GCP)
Passo 2	Criar a estrutura do repositório, Nginx local e proxy Caddy.	30 min	Nenhum
Passo 3	Fazer o upload dos arquivos de backup (gcp-prod-*) e scripts.	15 min	Nenhum
Passo 4	Executar a restauração de teste preliminar na Hostinger.	15 min	Nenhum
Passo 5	Ativar o “modo manutenção” no GCP e tirar o backup incremental final do banco.	15 min	 Downtime Temporário
Passo 6	Importar o dump final na Hostinger e atualizar o apontamento de DNS do domínio.	30 min	 Downtime Temporário
Passo 7	Configurar o Caddy para emissão automática do certificado SSL e testar acessos.	15 min	Fim da Janela de Manutenção

Etapa	Ação Técnica	Tempo Est.	Impacto na Operação
Passo 8	Instalar e configurar o Rclone para backup offsite no Google Drive da CDC e ativar o Cron.	20 min	Nenhum (sistema já ativo)

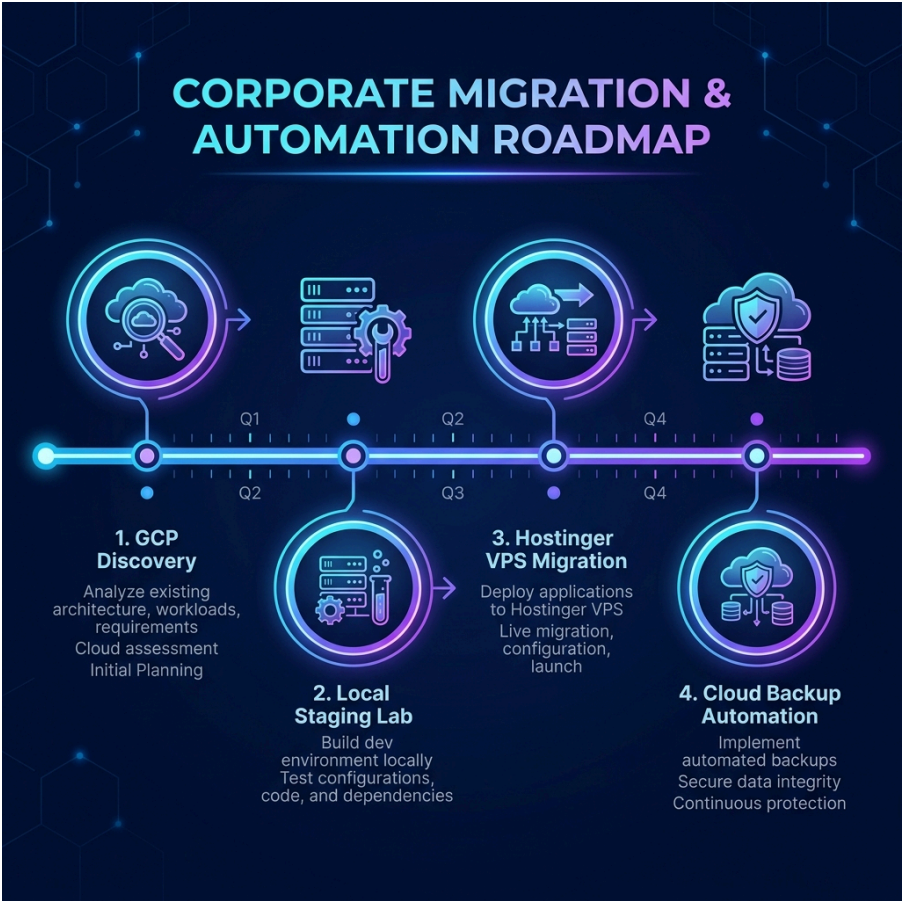


Figura 3: Roadmap com a Linha do Tempo e Etapas Restantes da Migração.

7. Plano de Melhoria Contínua para o Ambiente

Após a conclusão da migração para a Hostinger, propomos a implementação das seguintes melhorias de segurança e governança de TI na CDC:

- 1. Remoção de Senhas em Texto Puro (Hardcoded):**
 - Substituir a senha padrão do banco de dados (atualmente "admin") por chaves criptográficas fortes armazenadas no arquivo seguro `.env`.
- 2. Containerização Completa do Extrator de Dados:**
 - Empacotar os scripts do integrador em um contêiner Docker dedicado (com consumo limitado de CPU e memória RAM), eliminando tarefas soltas no servidor.

3. Otimização de Performance do Extrator:

- Otimizar o script de produtos (4_Extrator_produtos_v2.py), reduzindo seu tempo de execução de 4 minutos para menos de 30 segundos através de processamento em lote.

4. Notificações de Alerta de Backups no Mattermost:

- Conectar os scripts de backup ao canal de TI no Mattermost para notificar a equipe automaticamente sobre o sucesso ou qualquer falha nos backups diários.

5. Criptografia Assimétrica de Backups (GPG):

- Criptografar os arquivos de backup localmente antes do envio para a nuvem, garantindo que os dados fiquem protegidos contra vazamentos no Google Drive.

6. Substituição de Scripts Manuais pelo Rclone:

- Adotar o **Rclone** como ferramenta oficial do *CDC Backups Hub*, garantindo uploads automáticos, renovação transparente de chaves OAuth e suporte centralizado para os demais sistemas da instituição (ERPNext, Moodle, etc.).

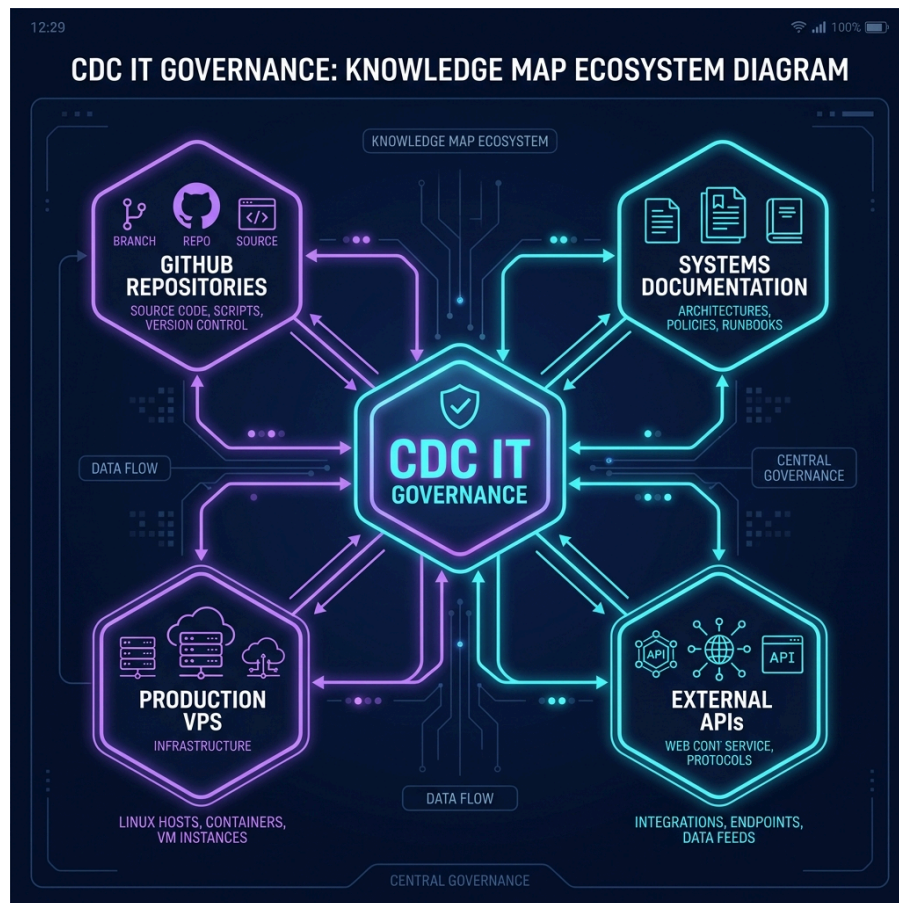


Figura 4: Mapa de Conhecimento e Governança de TI Conectando Sistemas e Documentações da CDC.